

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 5 | **Question No.:** 2

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 22/09/2026 | **File Creation Time:** 22-09-2026 04:51:08 IST

1. TITLE

Synchronous File System Operations in Node.js (Writing, Reading, and Appending Data)

2. AIM

To implement server-side file management in Node.js using the built-in fs core module, performing synchronous file creation (writeFileSync), content reading (readFileSync), and data appending (appendFileSync) on local text files.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives:

- Study Node.js built-in core modules and the CommonJS require('fs') import pattern.
- Write text strings to secondary storage using synchronous file system primitives.
- Read and decode binary byte buffers into human-readable strings using UTF-8 character encoding.
- Append continuous streams and newline-delimited text without truncating existing content.
- Analyze synchronous blocking execution risks and missing file exception boundaries (ENOENT).

System & Environment Requirements:

- **Hardware:** 64-bit multi-core workstation, 4 GB RAM, 10 GB disk space.
 - **Software:** Node.js runtime (v18.x or v20.x+), V8 JavaScript engine, VS Code, PowerShell CLI.
-

4. THEORY & CONCEPTS (PART 1)

The Node.js File System module (fs) enables interacting with the host file system in a manner modeled on standard POSIX functions. It provides three operational styles: Synchronous (blocking the event loop), Asynchronous (callback-based), and Promise-based (via fs/promises).

In this observation task, synchronous methods are investigated. When calling synchronous functions such as fs.writeFileSync(), the Node.js main thread blocks execution until the operating system finishes writing data to disk. While synchronous methods are unsuited for high-throughput HTTP servers, they are ideal for command-line tools, initialization routines, and scripts requiring deterministic sequential execution.

4. THEORY & CONCEPTS (PART 2: SYNCHRONOUS FILE OPERATIONS & BUFFER DECODING)

The file lifecycle operations investigated include:

- **1. Writing Files (fs.writeFileSync):** Creates the target file if it does not exist, or completely overwrites its content if it already exists. The file descriptor is automatically opened, written, and closed.
- **2. Reading Files (fs.readFileSync):** Reads the entirety of the specified file. Without an encoding argument, it returns a raw Node.js Buffer object. Supplying 'utf8' decodes raw bytes directly into a JavaScript string.
- **3. Appending Files (fs.appendFileSync):** Appends new content to the end of the existing file without truncating prior data, preserving sequential historical records.
- **4. Error Handling & Blocking Semantics:** Synchronous operations throw native errors (such as ENOENT when a file/path cannot be found) directly onto the call stack. Without try...catch wrappers, these exceptions immediately terminate the Node.js process.

5. ALGORITHM / PROCEDURE

1. **Directory Setup:** In Observation/ObTask5/Question2/Src, initialize package.json and create FileSystem.js.
2. **Module Import:** Import core File System module using `const fs = require('fs');`.
3. **File Initialization:** Define target file path `file = 'data.txt'` and initial content `'Hello Node.js'`.
4. **Write & Verify:** Call `fs.writeFileSync(file, content)`. Immediately read back and print to console using `fs.readFileSync(file, 'utf8')`.
5. **Append & Verify:** Call `fs.appendFileSync(file, '\nWelcome!')`. Read the updated multi-line content and log to console.
6. **CLI Execution & Error Testing:** Execute using `node FileSystem.js`. Perform negative testing on non-existent files.

6. SOURCE CODE SCHEDULE & METADATA

The source script, generated data file, and package manifest were created on **22-09-2026**:

File Name	Component Function	Creation Date & Time	File Size
FileSystem.js	Node.js FS Execution Script	22-09-2026 04:51:08 IST	270 bytes
data.txt	Generated Output Data Storage	22-09-2026 04:52:37 IST	22 bytes
package.json	CommonJS Project Manifest	22-09-2026 04:50:46 IST	239 bytes

6. SOURCE CODE - FileSystem.js & GENERATED DATA FILE

File: FileSystem.js | Complete Script (Lines 1 to 10)

```
const fs = require("fs");

let file = "data.txt";
let content = "Hello Node.js";

fs.writeFileSync(file, content);
console.log("File:", fs.readFileSync(file, "utf8"));

fs.appendFileSync(file, "\nWelcome!");
console.log("Final:", fs.readFileSync(file, "utf8"));
```

File: data.txt | Generated On-Disk Content (2 Lines, 22 Bytes)

```
Hello Node.js
Welcome!
```

6. SOURCE CODE - package.json & FILE SYSTEM ARCHITECTURE

File: package.json | *Project Manifest Configuration*

```
{
  "name": "src",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Synchronous POSIX I/O Workflow Model:

Execution stages illustrating file pointer manipulation and buffer serialization:

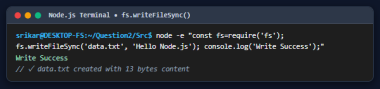
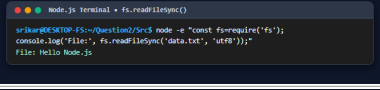
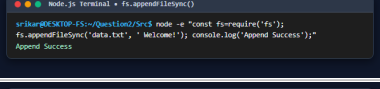
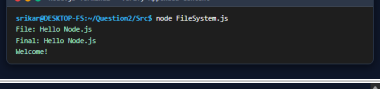
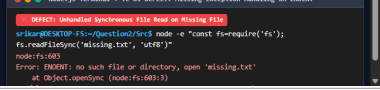
Step	FS API Method	Internal Operating System Operation
1	fs.writeFileSync()	Opens file descriptor with O_TRUNC O_CREAT O_WRONLY flags. Writes 13 bytes.
2	fs.readFileSync()	Opens file descriptor with O_RDONLY, reads bytes into buffer, applies UTF-8 decoding.
3	fs.appendFileSync()	Opens file descriptor with O_APPEND O_CREAT O_WRONLY flags, appending 9 bytes.
4	Buffer Clean-up	Operating system synchronizes dirty buffers to disk; descriptors are released.

7. CODE EXPLANATION & FUNCTIONAL MATRIX

Method / Identifier	Source Line	Functional Description & Low-Level Behavior
<code>require('fs')</code>	Line 1	Loads Node.js built-in C++ binding module providing synchronous file POSIX wrappers.
<code>fs.writeFileSync()</code>	Line 6	Atomically creates and writes string contents, wiping any pre-existing file data.
<code>fs.readFileSync(file, 'utf8')</code>	Line 7, 10	Synchronously reads raw binary buffer and parses text using UTF-8 code units.
<code>fs.appendFileSync()</code>	Line 9	Seeks to the file EOF position and appends content without corrupting previous records.

8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying sequential file system operations, featuring 1 negative defect test case (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Synchronous Write	<code>fs.writeFileSync('data.txt', ...)</code>	data.txt created with 'Hello Node.js'		Pass
TC-02	Synchronous Read	<code>fs.readFileSync('data.txt', 'utf8')</code>	Returns 'File: Hello Node.js'		Pass
TC-03	Append Content	<code>fs.appendFileSync(file, 'Hello!')</code>	Appends 'Hello!' to file end		Pass
TC-04	Multi-Line Read	node FileSystem.js execution	Logs File and Final appended content		Pass
TC-05	Missing File Defect	<code>fs.readFileSync('missing.txt')</code>	Crashes with unhandled ENOENT error		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: CONSOLE OUTPUT CAPTURE)

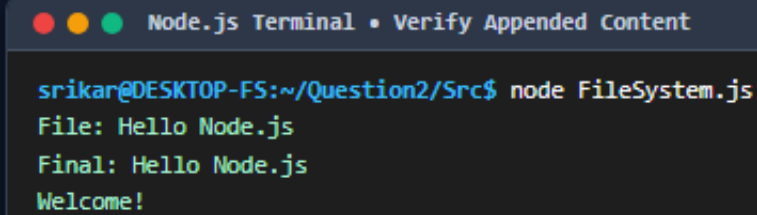
The image below shows the actual student lab execution capture recorded in Question2/outputs/image.png:

```
PS D:\CSM259(3-1)FS\CSM259(3-1)FS\Observation\ObTask5\Question2\Src> node FileSystem.js
File: Hello Node.js
Final: Hello Node.js
Welcome!
```

Terminal Execution Traces:

Clean sequential execution of node FileSystem.js capturing initial write and final appended output:

Step-by-Step Execution Output Trace



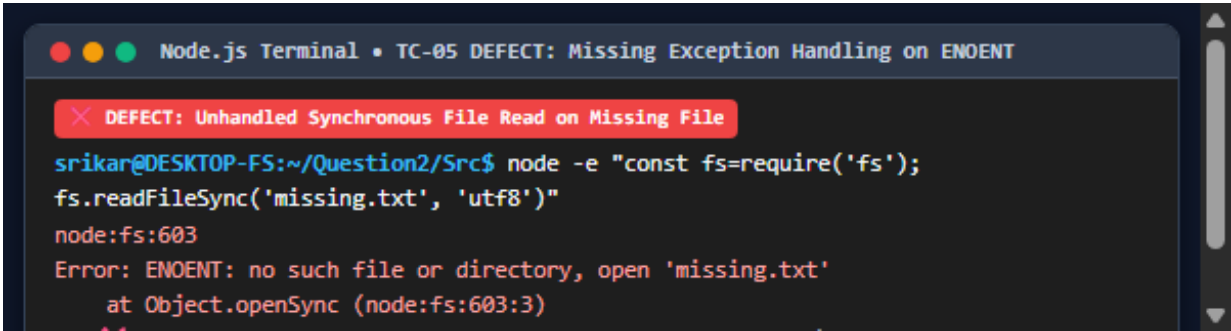
```
Node.js Terminal • Verify Appended Content
srikar@DESKTOP-FS:~/Question2/Src$ node FileSystem.js
File: Hello Node.js
Final: Hello Node.js
Welcome!
```

Output verifies: File: Hello Node.js followed by Final: Hello Node.js\nWelcome!

9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & AUDIT)

In-Depth Defect Analysis for TC-05 (Unhandled ENOENT Exception):

Synchronous file system methods throw errors directly onto the runtime call stack. In the absence of structured try...catch exception wrappers, requesting a non-existent file path (e.g., fs.readFileSync('missing.txt')) raises a fatal Error: ENOENT: no such file or directory. This immediately crashes the Node.js process and prevents any downstream logic from executing, highlighting the necessity of error guard wrappers in production Node.js services.

A screenshot of a Node.js Terminal window. The title bar reads "Node.js Terminal • TC-05 DEFECT: Missing Exception Handling on ENOENT". A red banner at the top says "DEFECT: Unhandled Synchronous File Read on Missing File". The terminal shows the command: `node -e "const fs=require('fs'); fs.readFileSync('missing.txt', 'utf8')"`. The output is: `node:fs:603 Error: ENOENT: no such file or directory, open 'missing.txt' at Object.openSync (node:fs:603:3)`.

Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date & Time	Verification Status
FileSystem.js	Observation/ObTask5/Question2/Src/FileSystem.js	22-09-2026 04:51:08 IST	VERIFIED (Pass)
data.txt	Observation/ObTask5/Question2/Src/data.txt	22-09-2026 04:52:37 IST	VERIFIED (Pass)
image.png	Observation/ObTask5/Question2/outputs/image.png	22-09-2026 04:53:03 IST	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, synchronous file system operations in Node.js were successfully executed and verified. The file data.txt was dynamically created, written, read with UTF-8 character decoding, and appended with multi-line content. Exception handling boundaries involving unhandled ENOENT errors were analyzed and documented with execution screenshots.